

Идентификация людей в реальном времени — на ноутбуке, без интернета, с 2023 года.

Narrow AI — модель решает одну задачу (обнаружение людей в кадре) и больше ничего.



Кадр модели в момент демо: 2 человека в боксах. YOLOv8 (Ultralytics, 2023). »

На экране — **YOLOv8** на CPU ноутбука: **~30 fps**.

Без интернета · обучена в 2023. ^[1]

[1] Ultralytics — YOLOv8 (2023) · [2] Google — MediaPipe

1 / 36

Сейчас вы видите простой эксперимент: ноутбук с веб-камерой через проектор показывает изображение зала, а небольшая программа в реальном времени обводит каждого присутствующего рамкой и считает людей в кадре. Под капотом — модель YOLOv8 [1] от Ultralytics, опубликованная в 2023 году. Она обучена на стандартном датасете COCO и распознаёт 80 классов объектов; для нас сейчас важен только класс «person». У этой демонстрации три значимых свойства, которые формируют один из главных тезисов лекции. Во-первых, модель работает локально — на CPU обычного ноутбука, без видеокарты и без обращения к облачным серверам. Во-вторых, скорость составляет около 30 кадров в секунду, что достаточно для реального времени на видеопотоке. В-третьих, никакого интернета: камера, модель и проектор — три компонента в одной комнате, никакие изображения никуда не передаются. Это пример того, что в литературе называют narrow AI — система, обученная на одной чётко поставленной задаче и больше ничего не умеющая. Та же модель не сможет вести с вами беседу, не напишет код, не предложит маркетинговую стратегию. Зато на своей единственной задаче она работает быстро и предсказуемо. Сегодня мы увидим, что таких систем в инженерной практике большинство, и что выбирать тип AI под задачу — отдельный навык, который мы будем тренировать весь курс. Источники: [1] Ultralytics — YOLOv8 (2023) — narrow-модель детекции; локальный inference ~30 fps на CPU без интернета. <https://docs.ultralytics.com/models/yolov8/> [2] Google — MediaPipe — on-device real-time ML pipelines — класс локальных narrow-решений. <https://ai.google.dev/edge/mediapipe/solutions/guide>

Добро пожаловать на курс «Отраслевое применение систем искусственного интеллекта»

17 лекций о том, где AI работает в индустриях — и где нет.

2 / 36

Здравствуйте. Это курс «Отраслевое применение систем искусственного интеллекта» — про то, как AI используется в реальных индустриях: от разработки ПО и финансов до авиакосмоса, сельского хозяйства и медицины. Курс не про то, как обучать модели с нуля, а про то, как грамотно выбирать, внедрять и оценивать готовые AI-решения в инженерном контексте. Сегодня — первая лекция, вводная. Начнём с короткой демонстрации, а потом перейдём к тому, зачем вообще нужен весь этот курс.

Главный вопрос курса — не «можно ли AI?», а «нужно ли и где?».

100% AI-пилотов запускаются

—90% откатываются

10% доходят
до прода

Иллюстрация принципа (Gartner, McKinsey подтверждают похожие цифры). »

Главная мысль курса

Завтра — почти везде.
Сегодня — почти никто.
Курс — про этот разрыв. ^[2]

Центральный вопрос курса

Где AI работает,
где — нет,
и как это понять?

[1] Gartner — пресс-релиз (окт 2024) · [2] Vedomosti / Intellectual Analytics (мар 2026)

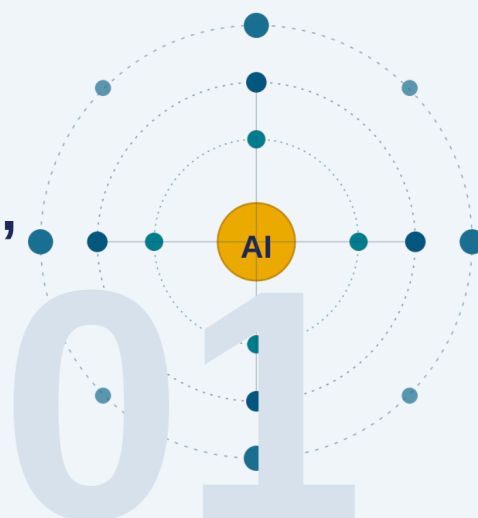
3 / 36

Вопрос «можно ли AI» в 2026 году уже неинтересен — можно. Девятьсот миллионов человек пользуются ChatGPT каждую неделю, половина разработчиков — каждый день. Это уже не «надо ли начинать», это уже инфраструктура, на которой работают другие продукты. А вот вопрос «нужно ли и где» — открытый. По сводным данным деловых изданий, около девяноста процентов корпоративных AI-пилотов [2] в России не доходят до полноценного промышленного развёртывания: тридцать-сорок процентов закрываются без эффекта, лишь семь-десять процентов доходят до production. Это не уникальная для России проблема — McKinsey и Gartner [1] регулярно публикуют похожие цифры по другим рынкам. Корпоративные AI-проекты повсеместно сталкиваются с разрывом между демонстрационной готовностью и продакшн-надёжностью. Это сочетание — массовое распространение и массовый провал — формирует центральную инженерную задачу курса. Главный вопрос звучит так: где AI работает, где не работает, и как заранее это понять. Это диагностический вопрос, не триумфальный. Курс не обещает рецепт успеха; он даёт способ отличать применения, в которых AI уместен, от применений, в которых он создаёт иллюзию решения. От этого различения зависят бюджет, время, надёжность системы и иногда — безопасность людей. К этому центральному вопросу мы будем возвращаться каждую лекцию курса. Источники: [1] Gartner — пресс-релиз (окт 2024) — 80% инженерных работников должны осваивать GenAI к 2027. <https://www.gartner.com/en/newsroom/press-releases/2024-10-03-gartner-says-generative-ai-will-require-80-percent-of-engineering-workforce-to-upskill-through-2027> [VFY-day-of] [2] Vedomosti / Intellectual Analytics (мар 2026) — 9 из 10 корпоративных GenAI-пилотов в РФ свёрнуты/отложены (n=50 крупных орг.). <https://www.vedomosti.ru/technology/articles/2026/03/24/1184974-biznes-svernul-ili-otlozhen-9-iz-10-proektov-po-vnedreniyu-generativnogo-ii> [VFY-day-of]

ЛЕКЦИЯ

Что такое AI? История, классификация, общие понятия

Карта применений AI: где работает, где — нет.



4 / 36

Это вводная лекция курса «AI в разных отраслях». Мы построим общую карту: что такое AI как класс технологий, в каком состоянии он находится в 2026 году, какие способы реализации систем с AI существуют в инженерной практике, и где у этих систем границы. К 2026 году AI перестал быть отдельным продуктом и стал инфраструктурным слоем, который незаметно работает в десятках сервисов, которыми вы пользуетесь каждый день. Голосовой ввод в смартфоне, навигация по пробкам, разблокировка лица, фильтр спама в почте, рекомендации в музыкальном сервисе, автодополнение в браузере — всё это AI, который большинство пользователей перестали воспринимать как AI. Цель лекции — не познакомить вас со всеми возможными AI-инструментами (это технически невозможно, рынок меняется быстрее, чем выходят учебники), а дать рабочий язык: чем модель отличается от чата, чем чат — от агента, и как за тридцать секунд понять, какой тип AI подходит под конкретную задачу. К концу лекции у вас будет простой инструмент — два диагностических вопроса плюс квадрант — который мы дальше будем применять во всех остальных лекциях курса.

План лекции



5 / 36

Прежде чем двигаться дальше — короткая навигация. Лекция состоит из пяти разделов, последовательно отвечающих на пять разных вопросов. Раздел 0, в котором мы сейчас, — открытие: короткая демонстрация, представление преподавателя и центральный вопрос курса. Раздел 1 — что такое AI и почему определений много. Здесь же пройдемся по семидесятилетней истории дисциплины, включая предысторию до появления самого термина, и зафиксируем ключевую точку перелома 2017 года. Раздел 2 — где мы сейчас, в 2022–2026 годах. Цифры проникновения AI в мире и в России, и обзор прорывов от не-первых игроков. Раздел 3 — самый большой раздел лекции. Четыре способа реализации систем с AI: модель, чат, агент, приложение. Разберём каждый со схемой и примером, а в финале раздела получим практический инструмент выбора типа AI. Раздел 4 — границы и безопасность. Куда уходят данные, что такое галлюцинации, три проявления подстраивания модели под данные, и концептуальные границы AI как класса технологий. Раздел 5 — заключение, карта семестра и анонс лекции 2.



**Левко Максим
Николаевич**

 **Telegram**
@Maxim_Levko

 **Email**
Levko.maxim@gmail.com

Кто я и почему мне это важно.

Архитектор, технический и продуктовый лидер создания
и внедрения информационных систем



20+ лет опыта в ИТ
10+ завершённых проектов
под руководством



Экспертиза
Системный анализ · Проектирование систем ·
Управление данными · Автоматизация бизнеса ·
Управление продуктами

Консалтинг и inhouse

 **Yandex**

 **МТС**

 **Магнит**

 **Сибур**

6 / 36

Для начала — коротко о себе. Я архитектор, технический и продуктовый лидер в разработке и внедрении информационных систем, больше двадцати лет в ИТ. За это время под моим руководством было реализовано больше десяти завершённых проектов — от системного анализа и проектирования архитектуры до управления данными, автоматизации бизнес-процессов и запуска продуктов. Я работал и как консультант, и как штатный сотрудник — с Яндексом, МТС, Магнитом, Сибуром. Это разные индустрии — технологии, телеком, ритейл, нефтехимия — и в каждой AI применялся по-своему, с разным успехом. Именно поэтому меня интересует не общий разговор про «искусственный интеллект», а конкретная инженерная граница: где AI реально экономит время и деньги, а где — красивая демонстрация, которая разваливается в продакшене. Этот курс — попытка дать вам карту, которой у меня самого не было, когда я только начинал разбираться с этими инструментами в реальных проектах. Мы будем разбирать конкретные инженерные решения, а не общие декларации, и границам — там, где AI ломается, — будет уделено столько же внимания, сколько успешным кейсам. Если в течение курса возникнут вопросы вне формата лекции — пишите в Telegram (@Maxim_Levko) или на почту (Levko.maxim@gmail.com). Чем раньше задаёте вопрос, тем полезнее будет ответ — не копите их до конца семестра.

Раздел 1 · Что такое AI

Определения, история, классификация.



7 / 36

Мы познакомились и обозначили главный вопрос курса. Дальше — первый содержательный раздел: что такое AI. Структура раздела короткая, но плотная. Сначала — почему определений AI много и откуда берётся эффект движущейся цели. Затем — факт-мостик: идея формального нейрона старше самого термина «искусственный интеллект». И в финале раздела — сжатый семидесятилетний таймлайн дисциплины с чёткой точкой перелома в 2017 году. К концу раздела у вас будет общий язык для разговора об AI: откуда взялись разные определения, почему они не противоречат друг другу, и в какой момент истории AI стал тем, чем является сейчас.

Определений AI много — потому что AI это движущаяся цель.

Russell & Norvig (AIMA, 2021)

«AI = система, мыслящая как человек, мыслящая рационально, действующая как человек или действующая рационально (4 квадранта по 2 осям).»

Russell & Norvig, AIMA, 4-е изд., 2021 »

ISO/IEC 22989:2022

«AI-система — это engineered system, которая генерирует выходы (рекомендации, прогнозы, решения) для целей, заданных человеком.»

Международный стандарт ISO/IEC 22989:2022 — опора EU AI Act »

Через обучение (Mitchell, 1997)

«Программа улучшается с опытом E на задаче T по метрике P. Если поведение возникает из обученной модели — это AI.»

Mitchell, Machine Learning, 1997 »

Через бенчмарки и AGI

«AI = то, что проходит тест Тьюринга или решает бенчмарк на уровне человека.» Возражение Сёрла: поведение ≠ понимание.

Тьюринг 1950 / Searle 1980 — Chinese Room »

AI Effect (Tesler): «AI is whatever hasn't been done yet». Как только техника начинает работать, её перестают называть AI.

[1] Russell & Norvig — AIMA, 4-е изд., (2021) · [2] ISO/IEC 22989:2022 · [3] Mitchell — Machine Learning (1997) · [4] Searle — Chinese Room (1980) / Turing (1950)

8 / 36

Любой студент, начавший читать литературу по AI, быстро обнаруживает, что одного канонического определения у предмета нет. Это не недоработка авторов учебников и не пробел в стандартизации — это отражение природы предмета. Прежде чем переходить к практическим типам систем, важно зафиксировать, какие определения сосуществуют в литературе и почему они не противоречат, а дополняют друг друга. Подход первый — Russell и Norvig в каноническом учебнике AIMA предлагают разложить определения AI по матрице: думание против действия, человекоподобно против рационально. Сами авторы склоняются к четвёртой клетке — «действующий рационально», потому что она не требует решать философский вопрос, что такое мышление. Подход второй — международный стандарт ISO/IEC 22989 [2], на который опирается европейская регуляторика. Определение через возможности и через источник целей: AI-система генерирует выходы для целей, заданных человеком. Подход третий — функциональный, через обучение. Программа улучшает поведение на задаче с метрикой за счёт опыта. Это инженерный критерий: если поведение системы можно полностью описать правилами «если — то», это не AI; если поведение возникает из обученной модели — это AI, даже если модель проста. Подход четвёртый — через сравнение с человеком и через бенчмарки. Породил линию тестов от шахмат до ARC-AGI. Классическое возражение — Chinese Room Сёрла [4]: бенчмарк-эквивалентность не означает понимания. И главное — AI Effect, описанный Памелой Маккордак: как только техника начинает работать, её перестают называть AI. Распознавание речи, навигация по пробкам, разблокировка лица — всё это в момент появления было AI, а сейчас называется просто функцией приложения. Поэтому определение AI всё время сдвигается: как только задача решена и встроена в продукт, она остывает до уровня обыденной техники. В курсе мы будем явно называть, какое из четырёх определений

работает в каждой ситуации. Главное — не выбирать одно как «правильное».

Источники: [1] Russell & Norvig — AIMA, 4-е изд. (2021) — 4 определения по 2 осям (думать/действовать × человекоподобно/рационально).

<https://aima.cs.berkeley.edu/> [2] ISO/IEC 22989:2022 — AI-система = engineered system для человеко-заданных целей; опора EU AI Act.

<https://www.iso.org/standard/74296.html> [3] Mitchell — Machine Learning (1997) — функциональное определение: поведение из обученной модели = AI.

<https://www.cs.cmu.edu/~tom/mlbook.html> [4] Searle — Chinese Room (1980) / Turing (1950) — AI Effect + возражение Сёрла: поведение на бенчмарке ≠ понимание.

<https://doi.org/10.1017/S0140525X00005756>

Идея нейросети старше самого термина «искусственный интеллект» на 13 лет.



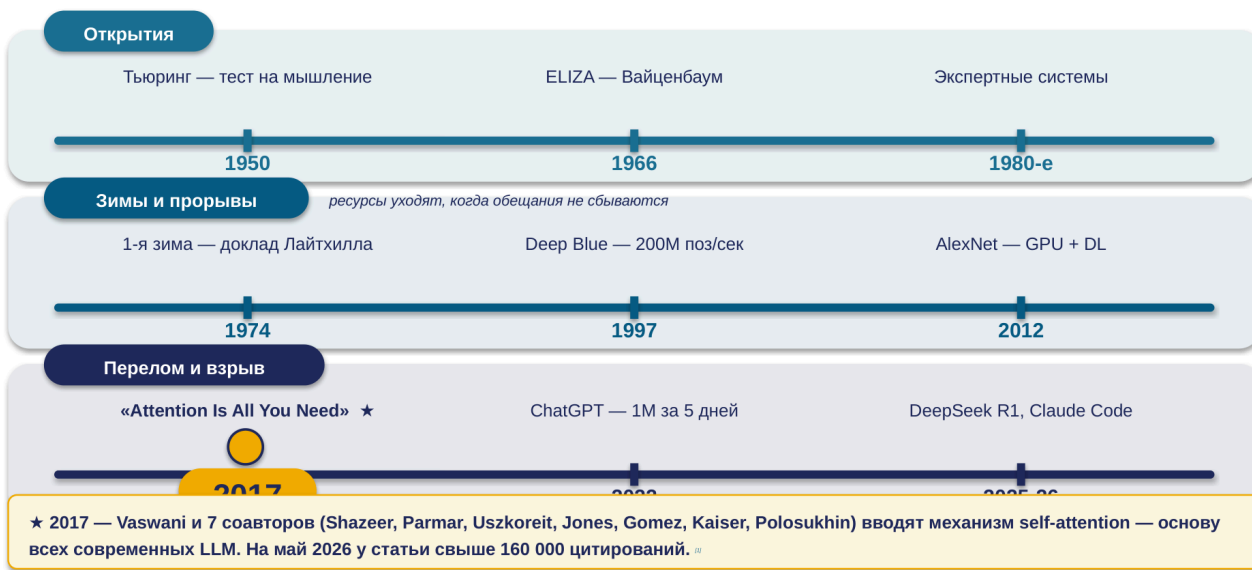
Идея «нейросети» на теоретическом уровне старше самого термина «искусственный интеллект».

[1] McCulloch & Pitts (1943)

9 / 36

Ещё до того, как появился сам термин «искусственный интеллект», нейрофизиолог Уоррен Мак-Каллок и логик Уолтер Питтс [1] опубликовали работу, формализующую нейрон как логический элемент: сеть таких простых бинарных элементов теоретически способна вычислить любую функцию, выразимую в терминах логики. Статья вышла в 1943 году — на 13 лет раньше Дартмутской конференции 1956 года, где термин «AI» появится впервые. Формальный нейрон Мак-Каллока — Питтса не был инженерной системой в современном смысле и не решал прикладных задач, но он предвосхитил коннекционистскую традицию — линию мысли, из которой десятилетия спустя вырастут искусственные нейронные сети и, в конечном счёте, архитектуры вроде Трансформера. Полезно держать в голове эту дату как напоминание: идея «нейросети» на теоретическом уровне старше самого термина «искусственный интеллект». Дальше мы посмотрим на историю уже с 1956 года — с момента, когда термин появился официально. Источники: [1] McCulloch & Pitts (1943) — формальный нейрон как логический элемент — на 13 лет раньше термина «AI». <https://doi.org/10.1007/BF02478259>

70 лет AI: открытия, зимы, точка перелома 2017. ^[2] ^[3]



[1] Vaswani et al. — Attention Is All You Need (2017) · [2] McCorduck — Machines Who Think (2004) · [3] Dhar — Paradigm Shifts in AI, CACM (2024)

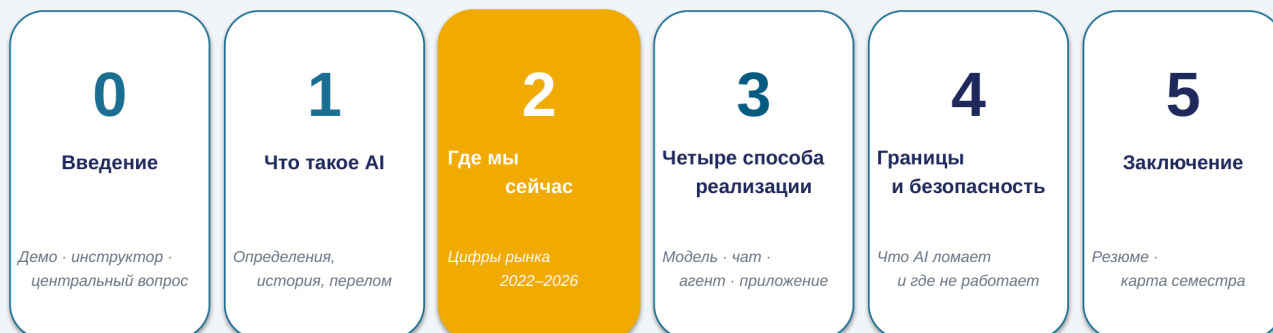
10 / 36

История AI как инженерной дисциплины — около семидесяти лет. Чтобы двадцать фактоидов не сливались, разделим хронологию на три тематические группы. Первая группа — открытия и первая практика, с пятидесятых до восьмидесятых. В пятидесятом году Тьюринг публикует «Imitation Game». В пятьдесят шестом — Дартмутская конференция, где Маккарти, Мински, Рочестер и Шеннон вводят сам термин «artificial intelligence». В шестьдесят шестом — Вайценбаум создаёт ELIZA, простую программу-психотерапевта, и обнаруживает, что пользователи приписывают ей понимание. В восьмидесятые — коммерческий бум экспертных систем: рулбейзед-системы XCON, MYCIN, японский проект «Пятого поколения». Вторая группа — зимы и прорывы, с семидесятых до начала десятых. Первая зима после доклада Лайтхилла британскому правительству и сокращения финансирования DARPA. Вторая — обрушение рынка специализированных AI-машин Symbolics в конце восьмидесятых. И прорывы между зимами: в девяносто седьмом Deep Blue от IBM побеждает Каспарова, оценивая двести миллионов позиций в секунду. В двенадцатом — AlexNet выигрывает ImageNet, доказывая, что глубокие свёрточные сети, обученные на GPU, бьют ручные признаки. Третья группа — точка перелома и взрыв. В семнадцатом году Васвани и семь соавторов публикуют «Attention Is All You Need», вводя архитектуру Трансформер [1]а. На май две тысячи двадцать шестого статья имеет более ста шестидесяти тысяч цитирований Google Scholar. В двадцать втором — OpenAI запускает ChatGPT; миллион пользователей за пять дней. В двадцать четвёртом — двадцать шестом — взрывной рост reasoning-моделей и агентских систем. Две зимы важны как методологический урок. Когда обещания массово не сбываются, поле теряет ресурсы. Сегодняшняя волна — четвёртая в истории AI. Пока избегает зимы за счёт того, что часть обещаний действительно сбывается. Но история учит: не все сбудутся, и инженер должен уметь различать то, что

работает уже сейчас, от того, что обещано на тридцатый год. Источники: [1] Vaswani et al. — Attention Is All You Need (2017) — Трансформер + self-attention; >160K цитирований (май 2026) — точка перелома. <https://arxiv.org/abs/1706.03762> [VFY-day-of] [2] McCorduck — Machines Who Think (2004) — AI Effect через исторические примеры остывания задач до «просто функции». https://www.google.com/books/edition/Machines_Who_Think/dPGij4vsHKgC [3] Dhar — Paradigm Shifts in AI, CACM (2024) — смена парадигм AI как рамка для чтения таймлайна. <https://doi.org/10.1145/3664804>

Раздел 2 · Где мы сейчас

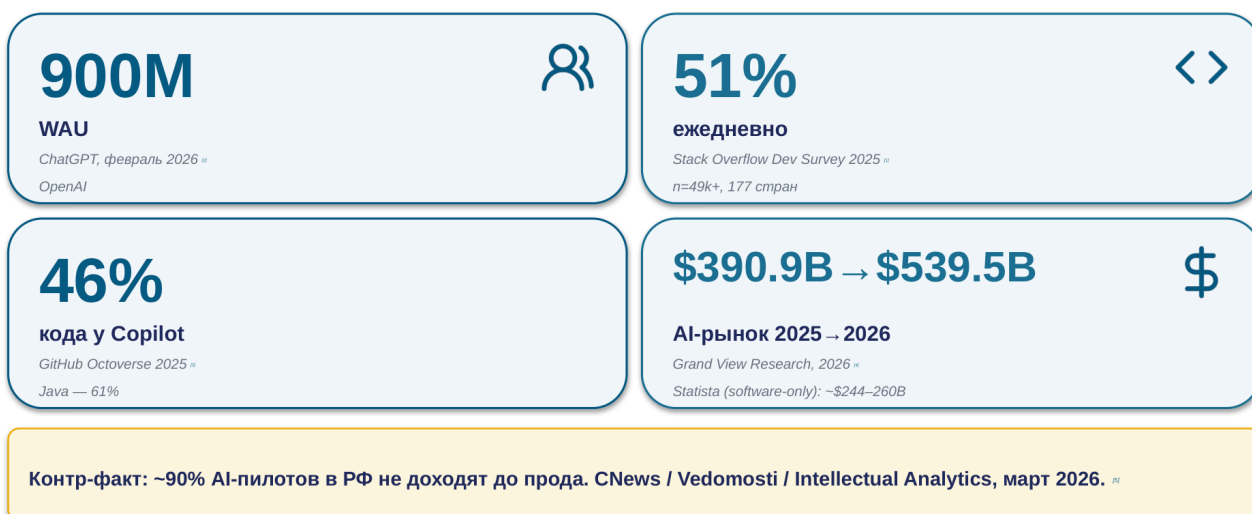
AI как инфраструктура, прорывы 2023–2026.



11 / 36

Мы разобрались, что такое AI и откуда он взялся. Дальше — короткий раздел про настоящее: где AI находится прямо сейчас, в 2026 году. Сначала — масштаб в цифрах: сколько людей и компаний уже используют AI ежедневно, и почему это одновременно уживается с массовым провалом пилотных проектов. Затем — обзор прорывов 2023–2026 годов от команд, которые не были очевидными фаворитами рынка: это открывает пространство для того, что дальше в курсе будем разбирать вы сами. Раздел короткий — шесть минут, — но именно он задаёт масштаб: AI одновременно везде и почти нигде не доведён до конца. Это напряжение — центральная тема всего курса.

AI стал инфраструктурой за 3 года: 900M пользователей, 51% разработчиков ежедневно, 46% Copilot-кода.



[1] Stack Overflow — Developer Survey 2025 · [2] OpenAI — ChatGPT WAU (фев 2026) · [3] GitHub — Octoverse 2025 · [4] Grand View Research (2026) · [5] Vedomosti / Intellectual Analytics (март 2026) · [6] Gartner — npecc-penux (окт 2024)

12 / 36

К двадцать шестому году AI перестал быть отдельным продуктом и стал инфраструктурным слоем, на котором работают другие продукты. Несколько показательных цифр. ChatGPT по состоянию на февраль 2026 имеет около девятист миллионов еженедельно активных пользователей [2]. Это weekly active users, не monthly: цифры месячной активности существенно выше. GitHub Copilot имеет более двадцати миллионов пользователей; у тех, кто им регулярно пользуется, до сорока шести процентов кода написано AI [3] [VFY], для языка Java — шестьдесят один процент. Stack Overflow Developer Survey 2025 [1], выборка сорок девять тысяч с лишним респондентов из ста семидесяти семи стран, показывает: восемьдесят четыре процента разработчиков используют или планируют использовать AI-инструменты в работе. Пятьдесят один процент профессиональных разработчиков работают с AI ежедневно. И одновременно сорок шесть процентов не доверяют точности AI-ответов на вопросы по коду — против тридцати одного процента годом ранее. Важная оговорка про доверие. Эти две цифры — пятьдесят один процент ежедневных пользователей и сорок шесть процентов недоверчивых — на первый взгляд кажутся противоречивыми. На самом деле они согласованы: общий тренд — рост проникновения, а точечная оценка точности кода стала критичнее, потому что разработчики накопили опыт работы с AI и научились видеть, где он ошибается. Это не «доверие падает», это «доверие становится более нюансированным». Дисциплина проверки — желательное поведение. Объём рынка — триста девяносто целых девять десятых миллиарда долларов в 2025 году с прогнозом роста до пятисот тридцати девяти с половиной миллиардов в 2026 (Grand View Research, 2026 [4]). По другим методологиям — например, узкому software-only определению Statista — оценки за 2025 год ниже, около 244–260 миллиардов. Разброс отражает то, что считать AI-рынком: продажи моделей, инфраструктуру, embedded-AI, услуги интеграции.

И параллельно — контр-факт. По сводным данным деловых изданий, около девяноста процентов AI-пилотов в России не доходят до полноценного промышленного развёртывания. Источник оценки — исследование консалтинговой компании Intellectual Analytics [5] (декабрь 2025 — февраль 2026, опубликовано Vedomosti в марте), выборка из пятидесяти крупнейших организаций в IT, промышленности, финансах, госсекторе и транспорте/логистике. Это не репрезентативная выборка по всей экономике; это срез по крупным игрокам, где AI-пилоты лучше всего задокументированы. Тем не менее, сочетание массового внедрения и массового провала формирует центральную инженерную задачу курса. Источники: [1] Stack Overflow — Developer Survey 2025 — n=49k+/177 стран; 51% профи ежедневно, 84% используют/планируют, 46% не доверяют коду. <https://survey.stackoverflow.co/2025> [2] OpenAI — ChatGPT WAU (фев 2026) — ~900M weekly active users — AI как инфраструктура. <https://techcrunch.com/2026/02/27/chatgpt-reaches-900m-weekly-active-users/> [VFY-day-of] [3] GitHub — Octoverse 2025 — adoption Copilot; ВНИМАНИЕ: «46% кода» этим отчётом НЕ подтверждается [VFY]. <https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/> [VFY-day-of] [4] Grand View Research (2026) — AI-рынок \$390.9B (2025) → \$539.5B (2026), CAGR 30.6%. <https://www.grandviewresearch.com/industry-analysis/artificial-intelligence-ai-market> [VFY-day-of] [5] Vedomosti / Intellectual Analytics (мар 2026) — контр-факт: ~9 из 10 пилотов РФ не доходят до прода (n=50). <https://www.vedomosti.ru/technology/articles/2026/03/24/1184974-biznes-svernul-ili-otl> ozhil-9-iz-10-proektov-po-vnedreniyu-generativnogo-ii [VFY-day-of] [6] Gartner — пресс-релиз (окт 2024) — 80% инженеров осваивают GenAI к 2027. <https://www.gartner.com/en/newsroom/press-releases/2024-10-03-gartner-says-generative-ai-will-require-80-percent-of-engineering-workforce-to-upskill-through-2027> [VFY-day-of]

Пространство открыто: 4 прорыва 2023–2026 от не-первых игроков.



Не отчаивайтесь: серьёзные прорывы делают разные команды. Курс — про устойчивые концепты, переживающие смену поколений моделей.

[1] Mistral AI — Announcing Mistral 7B (сен 2023) · [2] DeepSeek-R1 — mex.omx.com (яне 2025) · [3] SemiAnalysis — DeepSeek cost analysis (2025) · [4] Bloomberg — Nvidia \$589B drop (27 янв 2025) · [5] Steinberger — OpenClaw (GitHub) · [6] Gerganov — llama.cpp / ggml.ai

13 / 36

Глядя на цифры из предыдущего слайда, легко решить, что всё уже сделано: лидируют OpenAI, Google, Anthropic — крупнейшие игроки с многомиллиардными бюджетами. Это впечатление обманчиво. За последние три года в поле AI было несколько громких событий, каждое из которых показывает: пространство возможностей открыто, и серьёзные прорывы делают не только первые игроки. Урок: небольшая европейская команда может выпустить модель уровня крупных игроков за несколько месяцев от основания компании. Эпизод первый — Mistral 7B, сентябрь двадцать третьего года. Французская лаборатория Mistral AI, основанная в начале того же года выходцами из Meta и Google DeepMind, выпустила модель под лицензией Apache 2.0 — без ограничений на коммерческое использование. По бенчмаркам Mistral 7B превзошёл Llama-2 13B [1], модель почти вдвое большего размера. Урок: даже спорная методология подсчёта себестоимости может вызвать реальный, измеримый шок на глобальном рынке — проверяйте заявленные цифры, но не игнорируйте их эффект. Эпизод второй — DeepSeek R1, январь двадцать пятого. Китайская лаборатория выпустила reasoning-модель уровня OpenAI o1: девяносто семь и три процента на бенчмарке MATH-500 [2] против девяносто шести и четырёх у o1. Опубликованная стоимость одного финального тренировочного прогона V3 — около пяти и шести миллионов долларов, на порядки меньше западных оценок. Полная стоимость инфраструктуры по оценке SemiAnalysis [3] — один и три-один и шесть миллиарда. Это разные числа: стоимость одного тренировочного прогона и полная инфраструктура. Двадцать седьмого января рынок отреагировал — капитализация Nvidia упала на пятьсот восемьдесят девять миллиардов [4] долларов за один день, крупнейшая однодневная потеря капитализации в истории. Урок: один человек с правильным концептом может выпустить агент или продукт, меняющий рынок за недели. Эпизод третий — OpenClaw [5], ноябрь

двадцать пятого. Питер Стейнбергер, известный по Мас-разработке, в одиночку запустил open-source автономного AI-агента. Проект сменил несколько имён — Clawdbot, Moltbot — и закрепился как OpenClaw. К началу двадцать шестого года проект собрал больше ста тысяч звёзд на GitHub (из них шестьдесят тысяч за первые семьдесят два часа). Урок: один человек с правильным концептом может построить инфраструктурный слой, на который опираются тысячи — не продукт для конечного пользователя, а фундамент, на котором работает вся открытая экосистема. Эпизод четвёртый — llama.cpp [6] и ggml.ai, история Георгия Герганова. Проект начался как сольная инициатива Герганова — автора whisper.cpp — и вырос в маленькую независимую команду ggml.ai. Двадцатого февраля двадцать шестого года команда, вместо привлечения венчурного финансирования, присоединилась к Hugging Face за инфраструктурную поддержку, сохранив полную автономию и техническое лидерство: проект остался «на сто процентов open-source, community-driven, как и сейчас». В марте двадцать шестого llama.cpp пересёк сто тысяч звёзд на GitHub — быстрее, чем PyTorch или TensorFlow. К середине года — около ста семнадцати тысяч звёзд, около двадцати тысяч форков, вклад более семисот разработчиков. Не отчаивайтесь. Курс готовит вас к работе с устойчивыми инженерными концептами, которые переживают смену поколений моделей. Источники: [1] Mistral AI — Announcing Mistral 7B (сен 2023) — Apache 2.0, обходит Llama-2 13B — малая команда уровня лидеров. <https://mistral.ai/news/announcing-mistral-7b> [2] DeepSeek-R1 — тех.отчёт (янв 2025) — reasoning уровня o1; 97.3% MATH-500; спорная себестоимость. <https://arxiv.org/abs/2501.12948> [3] SemiAnalysis — DeepSeek cost analysis (2025) — полная инфра \$1.3–1.6B vs marginal train run V3 \$5.6M — разные числа. <https://semianalysis.com/2025/01/31/deepseek-debates/VFY-day-of> [4] Bloomberg — Nvidia \$589B drop (27 янв 2025) — крупнейшая однодневная потеря капитализации в истории. <https://www.bloomberg.com/news/articles/2025-01-27/asml-sinks-as-china-ai-startup-triggers-panic-in-tech-stocks> [VFY-day-of] [5] Steinberger — OpenClaw (GitHub) — соло open-source агент, 100K★ за квартал; rename-churn. <https://github.com/openclaw/openclaw> [VFY-day-of] [6] Gerganov — llama.cpp / ggml.ai — соло→HF (фев 2026), 100K+★ быстрее PyTorch/TensorFlow. <https://github.com/ggml-org/llama.cpp> [VFY-day-of]

Раздел 3 · Четыре способа реализации систем с AI

Не альтернативы, а слои.



14 / 36

Мы разобрались, что такое AI и где мы сейчас. Дальше — самый большой раздел лекции, двадцать три минуты. Четыре способа реализации систем с AI: модель, чат, агент, приложение. Структура раздела: сначала общая слоистая модель и классификатор «задача × модальность». Затем сравнение трёх способов через квадрант контроля. Затем — детальный разбор каждого типа со схемой и примером. И в финале — практический инструмент: два диагностических вопроса плюс квадрант, который вы будете применять на семинаре. Главная идея, которую держим в голове весь раздел: эти четыре типа — не альтернативы, а слои. Каждый следующий включает предыдущий и добавляет новые компоненты.

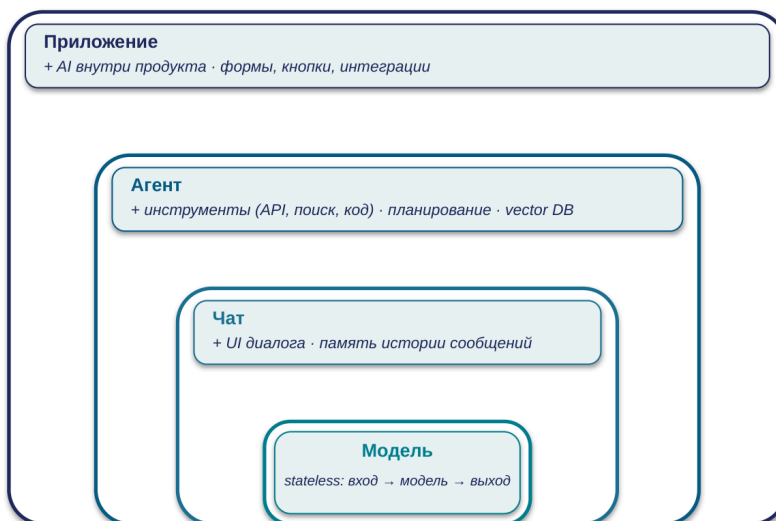
Способы реализации систем с AI: не альтернативы, а слои.

Каждый следующий слой

включает предыдущий

Не четыре альтернативные технологии, а четыре способа реализации одной задачи. « Каждое следующее обёртывание добавляет способности — и сложность, стоимость, потенциал ошибок. »

Выбор слоя — инженерное решение, не альтернатива.



[1] Anthropic — Building Effective Agents (2024) · [2] Weng — LLM Powered Autonomous Agents (2023)

15 / 36

Прежде чем разбирать каждый из четырёх типов реализаций, важно зафиксировать ключевую идею: это не четыре альтернативные технологии, а четыре способа реализации одной и той же задачи со слоистой [1]ой структурой. Каждый следующий уровень включает предыдущий и добавляет новые компоненты. Модель — самый базовый слой. Stateless-инференс на одной задаче. На вход данные, на выход предсказание. Без интерфейса, без памяти, без инструментов. Чат — следующий слой. Та же модель, но обёрнутая в текстовый интерфейс с памятью диалога в пределах сессии. Пользователь пишет сообщение, история обоих сообщений включается в контекст следующего обращения. Агент — ещё один слой. К чату добавляется возможность вызывать внешние инструменты — поиск, файловую систему, API, выполнение кода — и логика планирования последовательности действий. Приложение — внешний слой. Снаружи это обычный продукт со своими формами, кнопками, интеграциями. AI работает внутри как один из компонентов. Пользователь не пишет промпты, не общается с моделью напрямую. Эта вложенность важна потому, что многие практические недоразумения возникают из попытки выбрать между уровнями, как будто это альтернативы. На самом деле каждое следующее обёртывание — это инженерное решение, добавляющее способности и одновременно сложность, стоимость, потенциал ошибок. Чем выше уровень обвязки, тем больше места для багов между AI-компонентом и его окружением. В курсе мы будем многократно возвращаться к этой слоистой модели. Когда мы разбираем YOLO-детектор на конвейере, мы остаёмся на уровне модели. Когда мы используем ChatGPT для аналитики — на уровне чата. Когда строим агента, читающего двести PDF — на уровне агента. Когда говорим про Notion AI или YandexGPT в Поиске — про приложения, в которых AI лишь один из компонентов. Источники: [1] Anthropic — Building Effective Agents (2024) — слоистая эскалация:

простейшее решение → наращивать при необходимости.

<https://www.anthropic.com/research/building-effective-agents> [2] Weng — LLM Powered Autonomous Agents (2023) — Agent = LLM + Memory + Planning + Tool Use — верх
слоистой модели. <https://lilianweng.github.io/posts/2023-06-23-agent/>

Классификация AI-систем ^[1] — две оси: тип задачи × модальность. ^[2]

	 Классиф.	 Распозн.	 Поиск	 Генерац.	 Прогноз
Текст	BERT, спам	spaCy NER	BM25	GPT-4o, Claude	GPT-4o, Claude
Изображ.	ResNet	YOLO	CLIP	DALL-E, MJ	прогноз кадра
Звук / видео	PANNs	Whisper	Shazam	ElevenLabs, Sora	прогноз видео
Структ. данные	XGBoost	OCR таблиц	vector DB	Codex tab.	Prophet, ARIMA

YOLO — детекция объектов из демо в начале лекции. Подход к обучению и архитектура — позже в курсе.

[1] Russell & Norvig — AIMA (2021) · [2] Goodfellow, Bengio, Courville — Deep Learning (2016)

16 / 36

Чтобы говорить о конкретных AI-инструментах общим языком, нам нужна простая классификация [1]. В курсе мы используем две оси: тип задачи и модальность. Этих двух осей достаточно, чтобы расположить любой AI-инструмент, с которым мы будем работать в дальнейших лекциях. Более глубокая техническая классификация — по подходу к обучению и архитектуре нейросети — для нашего курса не основная цель этой лекции; глубже мы возвращаемся к этим темам на лекции 2. Ось «тип задачи» — что система производит на выходе. Классификация — поставить метку из конечного множества: спам или не спам, дефект или не дефект. Распознавание — обнаружить и идентифицировать что-то во входе: лица, речь, символы. Поиск — найти и отранжировать релевантные элементы: по корпоративной базе знаний, по фотографии. Генерация — создать новое содержимое: текст, изображение, аудио, код. Прогноз — предсказать число или последовательность: спрос, отказ оборудования, цену акции. Ось «модальность» — с каким типом данных работает система. Текст. Изображение — фотографии, рентгеновские снимки, кадры конвейерной камеры. Звук — речь, музыка, промышленный шум. Видео — последовательность изображений, часто плюс звук. Структурированные данные — таблицы, временные ряды, графы, базы. В ячейке «Поиск × Структурированные данные» стоит vector DB — база данных для поиска по смысловой схожести, подробнее разберём на слайде про агента. Мультимодальные комбинации — текст плюс изображение, текст плюс аудио плюс видео. Возьмём три примера, чтобы зафиксировать. Google Translate — задача генерация, модальность текст. AlphaFold — задача прогноз, модальность структурированные данные. YOLO, который вы видели в начале лекции — задача распознавание, модальность изображение. Эта простая таблица предотвращает смешение разных по природе систем под общим маркетинговым ярлыком «AI». Когда дальше в разделе мы будем разбирать типы реализаций, мы будем

спрашивать: какая задача и какая модальность. Источники: [1] Russell & Norvig — AIMA (2021) — классификация AI-систем как рабочий язык курса.
<https://aima.cs.berkeley.edu/> [2] Goodfellow, Bengio, Courville — Deep Learning (2016) — модальности и типы задач в терминах DL. <https://www.deeplearningbook.org/>

Одна задача, три способа: контроль распределяется между разработчиком и пользователем.



17 / 36

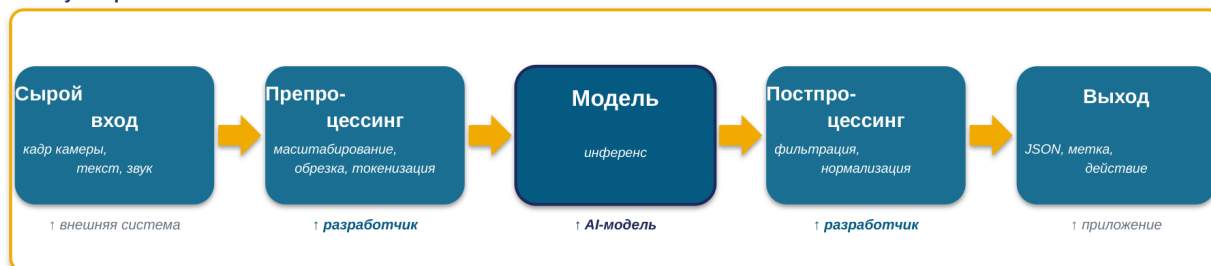
Одну и ту же задачу можно решить тремя разными способами реализации, и каждый из них по-разному распределяет контрол [1] между разработчиком и пользователем. Чем больше пользователь делегирует AI решение задачи, тем больше нужен жёсткий каркас от разработчика — поэтому Агент окажется в правом верхнем углу квадранта. Возьмём задачу: извлечь поля из входящего PDF-договора и положить в таблицу. Дата подписания, контрагент, сумма, срок действия. Пять-шесть полей. Способ первый — модель. Берёте предобученную или тонко настроенную модель извлечения структурированных полей из документов, например DocVQA или специализированную pougat-style модель. Пользователь сам интегрирует API: сам передаёт PDF, сам получает JSON, сам кладёт в свою таблицу. Делегирование от пользователя низкое — он сам контролирует каждый шаг. Контроль разработчика тоже низкий — модель это сырой inference-эндпоинт без оркестрации, без guard rails, без инструментов. Это левый нижний угол квадранта: низкое делегирование по горизонтали, низкий контроль разработчика по вертикали. Способ второй — чат. Открываете ChatGPT или YandexGPT, прикладываете PDF, пишете «извлеките следующие поля и верни в виде таблицы». Пользователь делегирует задачу словами и может уточнять, но он работает в рамках продукта — средняя степень делегирования. Разработчик задал системный промпт, базовые ограничения, контекстное окно — средний контроль с его стороны. Это центр квадранта. Способ третий — агент. Пользователь говорит «сделай эту задачу» и одобряет ключевые узлы — высокая степень делегирования, пользователь не управляет промежуточными шагами. Разработчик собрал агента: оркестратор планирует шаги, инструменты — открыть PDF, прогнать OCR, выделить поля, записать в таблицу, у агента есть память между шагами и возможность делать несколько вызовов. Высокий контроль разработчика — он спроектировал весь decision loop, список tools, guard

rails и system prompt. Это правый верхний угол квадранта: высокое делегирование по горизонтали, высокий контроль разработчика по вертикали. Два пустых угла объясняют, почему расположение неслучайно. Левый верхний (высокий контроль разработчика, но пользователь не делегирует) — нет смысла: зачем строить оркестратор, если пользователь сам всё решает по шагам. Правый нижний (полное делегирование без guard rails) — опасная зона: пользователь отдал задачу AI, а разработчик не предусмотрел рамок. Поэтому диагональ снизу-слева вверх-направо — естественная траектория: больше делегирования требует больше каркаса. Это распределение контроля — не достоинство одного способа над другим. Это инженерное решение: где должен лежать контроль для конкретного сценария. Если важна гибкость пользователя — модель. Если задача типовая, но требует уточнений — чат. Если поток большой и нужна автоматизация — агент. На семинаре будем тренировать этот выбор. Источники: [1] Anthropic — Building Effective Agents (2024) — распределение контроля разработчик↔пользователь по мере делегирования.
<https://www.anthropic.com/research/building-effective-agents>

МОДЕЛЬ

Модель — компонент, не система. Инференс: вход → препроцессинг → модель → постпроцессинг → выход.

Это уже приложение



Препроцессинг и постпроцессинг — ответственность разработчика. * YOLO = 50 строк; рабочая система с YOLO = сотни строк.

[1] Kreuzberger et al. — MLOps (2023) · [2] Jumper et al. — AlphaFold, Nature (2021)

18 / 36

Модель в нашей терминологии — это обученная нейросеть или другой ML-алгоритм, которая принимает вход определённого типа и возвращает выход. У неё нет ни состояния между вызовами, ни доступа к внешним инструментам, ни диалога с пользователем. С точки зрения инженерной интеграции это самый простой компонент: вызов модели — это функция, и она ведёт себя как функция. Очень важная деталь, которую часто упускают: модель — это не самостоятельная система, а компонент. На production вокруг модели всегда стоит обвязка из препроцессинга и постпроцессинга. Схематично: сырой вход — например, кадр с камеры, текст или аудио — проходит через препроцессинг (ресайз, нормализацию, токенизацию), затем через инференс модели, затем через постпроцессинг (фильтрация, форматирование, упаковка bounding box, softmax), и только тогда получается выход для приложения. За препроцессинг и постпроцессинг отвечает разработчик системы, не сама модель. Это не оговорка — это критически важно для оценки бюджета и сложности AI-проекта. YOLO-детектор сам по себе занимает условные пятьдесят строк кода, но рабочая система с YOLO внутри содержит сотни строк: считывание видеопотока, ресайз кадров, нормализация, NMS-фильтрация после инференса, упаковка результата в формат, который ожидает MES-система завода. Оценка проекта «вкатать YOLO на линию» по объёму работы модели даёт заниженную оценку в десятки раз. Канонические примеры моделей. YOLO — детекция объектов на изображениях, тот самый, что был в начале лекции. Whisper от OpenAI — распознавание речи. Stable Diffusion — генерация изображений из текста. BERT — векторные представления текста. AlphaFold [2] — предсказание трёхмерной структуры белка по аминокислотной последовательности; за неё Хассабис и Джампер получили Нобелевскую премию по химии 2024 года. Segment Anything от Meta — универсальная сегментация изображений. Когда модель — правильный выбор.

Высокая нагрузка, например обработка видеопотока тридцать кадров в секунду — невозможно для чат-интерфейса. Одна чётко определённая задача со стабильной формой входа и выхода. Требование детерминированности или предсказуемости. Edge-развёртывание без интернета. Встраивание AI как функциональной единицы внутрь большего продукта. Источники: [1] Kreuzberger et al. — MLOps (2023) — pre/post-processing вокруг модели — ответственность разработчика. <https://arxiv.org/abs/2205.02302> [2] Jumper et al. — AlphaFold, Nature (2021) — канонический пример модели-прогноза; Нобель по химии 2024. <https://doi.org/10.1038/s41586-021-03819-2>

ЧАТ

Как работает чат: цикл диалога.



Чат — это конвейер «собрать → подать → дописать → показать».

[1] Anthropic — Building Effective Agents (2024)

19 / 36

Чат удобно обсуждать на бытовом уровне: задал вопрос — получил ответ. Но за этим простым опытом стоит вполне конкретный технический цикл. Понимание этого цикла снимает магию и даёт два практических навыка: как контролировать чат через системный промпт [1] и как объяснить, почему чат «забывает» старые сообщения. Шесть шагов цикла. Первый — пользователь вводит текст в окно чата: вопрос, инструкцию, фрагмент данных. Второй — система берёт введённый текст и присоединяет его к системному промπτ. Системный промпт — это заранее заданные разработчиком инструкции: «Ты корпоративный ассистент компании X. Отвечай на русском. Не давай юридических советов. Если не знаешь — скажи "не знаю"». Дополнительно подмешивается вся текущая история чата с начала сессии. Третий — полный собранный пакет, то есть системный промпт плюс история плюс новое сообщение, отправляется в LLM как один большой текст. Модель не «помнит» прошлый разговор — она каждый раз получает его заново как часть входа. Четвёртый — модель генерирует ответ токен за токеном на основе вероятностного распределения следующего токена при заданном контексте. Пятый — ответ модели дописывается в историю чата на стороне системы. С этого момента он часть контекста для всех следующих обращений в этой же сессии. Шестой — ответ показывается в интерфейсе чата. Пользователь читает, отвечает, и цикл повторяется с шага один. Но теперь история уже больше, и LLM при следующем обращении получит весь набор: системный промпт плюс предыдущие пары вопрос-ответ плюс новое сообщение. Из этого цикла два практических следствия. Первое — системный промпт это инженерный рычаг. Через него разработчик задаёт рамки чата: тон, роль, ограничения, обязательные форматы ответа. Один и тот же ChatGPT в роли корпоративного ассистента и в роли обычного помощника ведёт себя по-разному именно из-за системного промпта. Второе — у каждой модели есть контекстное окно: максимальное число

токенов за один запрос. Для современных моделей это десятки и сотни тысяч токенов. Когда история чата перестаёт помещаться в окно, старые сообщения выпадают. Поэтому чат «забывает» детали из начала длинного разговора.

Источники: [1] Anthropic — Building Effective Agents (2024) — системный промпт как инженерный рычаг; контекст собирается заново каждый шаг.

<https://www.anthropic.com/research/building-effective-agents>

ЧАТ

Чат = модель + интерфейс + память диалога.

Кейс — типовой для чата

Инженер получил непонятное ТЗ от смежного отдела, нужно составить чек-лист своей работы.

Вы: Объясни пункт 4.2 ТЗ простыми словами.

Чат: Это требование к...

Вы: Составь чек-лист на 5 пунктов.

Чат: 1. Проверить... 2. Согласовать... 3. ...

Разовая задача с уточнениями — оптимально для чата. Не модель, не агент,

Оговорка для промышленных систем

Чистые чаты почти не используются в промышленной эксплуатации.

Почти везде они расширены до агентов «я» — хотя бы для долгосрочной памяти и поиска по корпоративной базе (RAG).

Архитектуру агента разберём на следующем слайде.

Выбор чата — точка на шкале взаимодействия, не единственно верный вариант.

[1] ВЦИОМ — «Нейросети в жизни россиян» (окт 2025) · [2] Dam et al. — Survey on LLM-based AI Chatbots (2024)

20 / 36

Чат — это модель, обёрнутая в текстовый интерфейс с памятью диалога в пределах сессии. Пользователь пишет сообщение, модель отвечает, история обоих сообщений включается в контекст следующего обращения. Это даёт ощущение разговора и позволяет уточнять задачу пошагово, что радикально меняет паттерн использования. Современные чаты — это в первую очередь LLM-чаты, хотя бывают и мультимодальные: с возможностью передавать изображения, аудио, файлы. Канонические продукты вы знаете: ChatGPT, Claude, Gemini, DeepSeek Chat, GigaChat, YandexGPT, Mistral Le Chat. Между ними различия в качестве для разных задач, в длине контекстного окна, в стоимости, в доступности модальностей — но базовый паттерн взаимодействия один. Возьмём конкретный пример. Инженер получает непонятный нормативный документ — скажем, ТЗ от смежного отдела с массой ссылок на стандарты и сложными формулировками. Задача — разобраться и составить чек-лист для собственной работы. Это типовой кейс для чата: разовая задача, требующая диалога, с уточнениями по ходу. Здесь не нужна отдельная специализированная модель, потому что нет потока однотипных задач. Не нужен агент, потому что не требуется автоматически открывать файлы и API. И нет специализированного приложения. Чат — оптимальный инструмент. Важная оговорка для промышленных систем. Чистые чаты в том виде, как они есть «из коробки» — то есть просто LLM плюс UI плюс короткая память сессии — в корпоративной среде почти не встречаются. Почти везде, где речь идёт о реальном внедрении, чат расширен до агента [2]: хотя бы для того, чтобы хранить долгосрочную память между сессиями, или для того, чтобы искать ответ в корпоративной базе знаний через RAG (retrieval-augmented generation). Архитектуру агента — что именно добавляется к чату — разберём на следующем слайде. Выбор чата — это инженерное решение о точке на шкале взаимодействия: где правильно выбран

тип взаимодействия с AI, там система работает предсказуемо. Источники: [1] ВЦИОМ — «Нейросети в жизни россиян» (окт 2025) — проникновение AI-чатов в РФ; 51% пользуются раз в неделю+.

<https://wciom.ru/analytical-reviews/analiticheskii-obzor/neiroseti-v-nashei-zhizni>

[VFY-day-of] [2] Dam et al. — Survey on LLM-based AI Chatbots (2024) — таксономия LLM-чатов; чистые чаты редки в проде (расширены до агентов).

<https://arxiv.org/abs/2406.16937>

АГЕНТ

Агент = чат + оркестратор + внешняя память + инструменты. [1] [2] [3]



ReAct (Reasoning + Acting) — Yao et al. 2022 (arXiv:2210.03629)

[1] Weng — LLM Powered Autonomous Agents (2023) · [2] Anthropic — Building Effective Agents (2024) · [3] Anthropic — Model Context Protocol (2024)

21 / 36

Агент — следующий слой после чата. По сравнению с чатом у агента появляется три дополнительных компонента, которых у чата не было. Первый компонент — оркестратор [2]. Логика управления: план, decision loop, критика. Оркестратор смотрит на цель пользователя, разбивает её на шаги, выбирает, какой инструмент применить на каждом шаге, и решает, когда остановиться. Второй — внешняя память. В отличие от чата, у агента есть память за пределами одной сессии: vector database (база данных, оптимизированная для поиска по сходству эмбеддингов; подробнее — позже в курсе), файловая система с промежуточными результатами, лог собственных предыдущих действий. Третий — инструменты. Возможность вызывать внешние API, читать и писать файлы, выполнять код, делать веб-поиск. Каждый инструмент — это функция с описанием: что делает, какие параметры принимает, что возвращает. Оркестратор выбирает, какой инструмент применить на каждом шаге. Цикл работы агента, базовая модель ReAct — Reasoning plus Acting. Шаг план: оркестратор и LLM формулируют план действий. Шаг act: выбирается и вызывается инструмент с конкретными параметрами. Шаг observe: результат инструмента наблюдается, записывается в память, передаётся обратно в LLM. Шаг reflect: LLM обдумывает результат — цель достигнута или нет, нужен ли следующий шаг. Если цель достигнута или итераций было слишком много — остановка. Иначе — следующий шаг.

Каноническая формулировка из работы Лилианы Венг: Agent равен LLM плюс Memory плюс Planning плюс Tool Use. Канонические продукты-агенты: Claude Code от Anthropic — агент в терминале для разработки. Devin от Cognition Labs — облачный коддинг-агент. OpenAI Operator — агент для веб-задач. AutoGPT — open-source автономный агент. Источники: [1] Weng — LLM Powered Autonomous Agents (2023) — Agent = LLM + Memory + Planning + Tool Use.

<https://lilianweng.github.io/posts/2023-06-23-agent/> [2] Anthropic — Building Effective

Agents (2024) — оркестратор + инструменты + внешняя память как слой над чатом. <https://www.anthropic.com/research/building-effective-agents> [3] Anthropic — Model Context Protocol (2024) — стандарт подключения инструментов/данных к агенту. <https://www.anthropic.com/news/model-context-protocol>

АГЕНТ

Агент за работой: 200 PDF — последовательность шагов.

Кейс — типовой для агента

200 PDF-отчётов.
Извлечь дату, контрагента, сумму.
Собрать сводную таблицу.

Не модель — нет специализированной модели
«возьми 200 произвольных PDF».

Не чат — некомфортно копировать 200
файлов в окно.

Агент — естественный выбор.

Что делает агент — пошагово, с указанием инструмента

- 1 Получить список файлов файловая система
- 2 Открыть PDF #1 чтение PDF
- 3 Извлечь текст извлечение текста (OCR / парсер)
- 4 Сделать сводку → vector DB эмбединги + векторная БД
- 5 Найти ключевые поля поиск + извлечение LLM
- 6 Записать строку в таблицу запись в таблицу (Sheets API / CSV)
- 7 Цикл по всем 200 файлам цикл оркестратора

Агент = последовательность вызовов инструментов, оркестрируемая LLM. ^[1]

[1] Yao et al. — ReAct (2022)

22 / 36

Кейс, на котором естественно появляется агент. У вас есть двести PDF-отчётов, и нужно из каждого извлечь определённые поля — дату, контрагента, сумму договора, — а потом собрать сводную таблицу. Это многошаговая задача с инструментами: открыть файл, прочитать, извлечь, сложить в таблицу. Она плохо подходит ни для модели — нет специализированной модели «возьми двести произвольных PDF и сделай таблицу». Ни для чата — некомфортно копировать двести файлов в окно. Агент — естественный выбор. Посмотрите на правую половину слайда. Я разбил работу агента на семь шагов, и для каждого явно показан инструмент, который агент использует. Это важно — агент не «думает» весь путь сам, он на каждом шаге решает: какой инструмент сейчас нужен, какие параметры в него передать, как обработать результат. Шаг один — получить список файлов. Инструмент: файловая система. Шаг два — открыть очередной PDF. Инструмент: PDF reader. Шаг три — извлечь текст. Инструмент: text extraction, через OCR если PDF сканированный, или через парсер если структурный. Шаг четыре — сделать сводку и положить эмбединг в vector DB. Шаг пять — найти ключевые поля по запросу: «найди дату подписания», «найди контрагента», «найди сумму». Это комбинация поиска и извлечения через LLM. Шаг шесть — записать строку в сводную таблицу через Sheets API или просто в CSV. Шаг семь — повторить для всех двухсот файлов; здесь работает оркестратор, который держит индекс текущего файла и общее состояние. Главное — агент это последовательность вызовов [1]ов конкретных инструментов, оркестрируемая LLM. Не «AI разобрался сам», а «AI вызвал нужные инструменты в правильном порядке». Источники: [1] Yao et al. — ReAct (2022) — Reasoning+Acting: план→действие→наблюдение→рефлексия с явным инструментом на шаге. <https://arxiv.org/abs/2210.03629>

АГЕНТ

Уровни автономии AI-агентов: дизайн-решение, не свойство модели.

5 уровней автономии (Feng / McDonald / Zhang, 2025) ^[1]

5. Наблюдатель (Observer)

Пользователь: только получает результат

AutoGPT на ночь

4. Утверждающий (Approver)

Пользователь: утверждает на контрольных точках

агент собирает PR, ждёт review

3. Консультант (Consultant)

Пользователь: ставит цель, правит план

Devin фиксирует баг по тикету

2. Соавтор (Collaborator)

Пользователь: работает наравне с агентом

Cursor парное программирование

1. Оператор (Operator)

Пользователь: одобряет каждое действие

Claude Code — approve each

Где находится человек по отношению к циклу

Человек вне цикла (Human-out-of-the-loop)

Человек только смотрит итог (≈ ур. 5).

Человек над циклом (Human-on-the-loop)

Человек наблюдает, прерывает при отклонениях (≈ ур. 3-4).

Человек в цикле (Human-in-the-loop)

Человек одобряет каждый шаг (≈ ур. 1-2).

Режимы ручного перехвата (Override)

Любой уровень — с ручным перехватом по тревоге.

Уровень автономии — выбор продукта, не свойство модели. На семинарах будем выбирать осознанно.

[1] Feng, McDonald, Zhang — Levels of Autonomy (2025)

Важная концепция, к которой мы будем возвращаться весь курс: уровни автономии агента. Фэн, Макдональд и Чжан [1] в работе двадцать пятого года предлагают пять уровней, характеризующих ролью пользователя. Не сложностью модели — именно ролью пользователя. Уровень один — Operator. Пользователь активно управляет каждым шагом, агент исполняет под одобрение. Пример: Claude Code в режиме «approve each command» — вы видите команду перед запуском и каждый раз нажимаете Enter. Уровень два — Collaborator. Пользователь и агент работают совместно, перетекая ролями. Пример: парное программирование с Cursor, где вы и AI пишете в одном файле и редактируете предложения друг друга. Уровень три — Consultant. Пользователь ставит цель, агент предлагает план и ведёт по шагам, пользователь корректирует. Пример: Devin получает тикет «исправь баг X», предлагает декомпозицию, вы поправляете один шаг, дальше Devin продолжает сам. Уровень четыре — Approver. Агент действует автономно, пользователь утверждает ключевые узлы. Пример: агент собирает PR, открывает его, ждёт вашего ревью; merge только после approve. Уровень пять — Observer. Агент работает полностью автономно, пользователь только наблюдает итог. Пример: AutoGPT запущен на ночь с заданием «собери конкурентный анализ» — вы возвращаетесь утром и читаете отчёт. Параллельно с этой пятёркой существует более старая и широко принятая в инженерии безопасности рамка: где находится человек по отношению к циклу. Human-in-the-loop — человек одобряет каждый шаг, это уровни один-два. Human-on-the-loop — человек наблюдает в реальном времени и прерывает при отклонении, это примерно уровни три-четыре. Human-out-of-the-loop — человек только читает итог, это уровень пять. И отдельно — override modes: на любом уровне можно сохранить возможность ручного перехвата по тревожным сигналам. Главная мысль слайда: уровень автономии — это дизайн-решение, а не

свойство модели. Один и тот же агент в режиме Operator и в режиме Observer — это разные продукты с разными требованиями к надёжности, объяснимости и интерфейсу. На семинарах будем возвращаться к этой шкале при выборе уровня под конкретный риск. Источники: [1] Feng, McDonald, Zhang — Levels of Autonomy (2025) — 5 уровней по роли пользователя:

operator→collaborator→consultant→approver→observer.

<https://arxiv.org/abs/2506.12469>

Приложение = AI, упакованный в продуктовый интерфейс.

Google Translate — масштаб 2026

1+ миллиард пользователей в месяц · **1 триллион** слов переведено / месяц

в Google Translate, Search, Lens и Circle to Search (Google Blog, апрель 2026) »



Google Translate

нейронная трансляция



Notion AI

GPT-4/Claude в кнопках



ЯндексGPT в Поиске

AI-краткий ответ



Grammarly

NLP + LLM подсказки



Яндекс.Карты

ML маршрутизация



Adobe Firefly

диффузия в Photoshop

AI как функция, а не продукт. Большинство студентов уже ежедневно пользуются всеми шестью. »

[1] Google — Translate at 20 (2026) · [2] Anthropic — Building Effective Agents (2024)

24 / 36

Приложение — это полноценный продукт для конечного пользователя, в котором AI лишь один из внутренних компонентов. Пользователь не пишет промпты; он нажимает кнопки, заполняет формы, говорит голосом. AI работает «под капотом», и от пользователя редко требуется понимать, что внутри. Базовая характеристика — AI как функция, а не продукт. Приложение решает доменную задачу, и AI обеспечивает соответствующую функциональность. У хорошего приложения детерминированный пользовательский интерфейс — то же действие пользователя даёт ту же реакцию, даже если AI-выход внутренне варьируется. Приложение добавляет вокруг AI ограничители, запасные сценарии, кэширование, ревью ошибок. Канонические примеры однозначных приложений. Google [1] Translate — внутри нейронная машинная трансляция; пользователь видит окно ввода и перевод. На двадцать шестой год — более одного миллиарда уникальных пользователей в месяц, около одного триллиона переведённых слов в месяц. Этот триллион считается across Google Translate, Search, Google Lens и Circle to Search, не только в самом приложении Translate. Notion AI — внутри GPT-4 или Claude; пользователь видит кнопки «Summarize», «Improve writing», «Continue writing» прямо в интерфейсе своего рабочего пространства. AI здесь не отдельный чат, а функциональная кнопка рядом с каждым блоком текста. YandexGPT в Яндекс.Поиске — внутри собственная LLM Яндекса; пользователь видит сгенерированный краткий ответ над списком обычных поисковых результатов. Это типичное «AI как функция»: для пользователя это просто улучшенный поиск, не «чат с AI». Grammarly — NLP-модели плюс LLM; пользователь видит подчёркивания и подсказки в любом текстовом поле. Яндекс Навигатор и Карты — ML-модели для маршрутизации, ETA-прогнозов, обнаружения пробок. Adobe Firefly — диффузионные модели, кнопка «Generate» в Photoshop. Кейс, для которого приложение — оптимальный выбор: вам раз в

неделю нужно перевести небольшой блок техдокументации с английского на русский. Не нужна standalone-модель — объём не оправдывает развёртывания. Не нужен чат — пользоваться чатом для перевода громоздко. Не нужен агент — задача однашаговая. Google Translate или ЯндексGPT в Поиске — готовые приложения, оптимизированные под именно этот тип использования.

Переплачивать сложностью — анти-паттерн. Важное уточнение перед

следующим слайдом. Приложения бывают двух типов. Первый — с пользовательским интерфейсом, как все шесть примеров на этом слайде:

пользователь видит UI и взаимодействует с ним. Второй тип —

автоматизированные приложения без интерфейса в момент исполнения:

ETL-pipeline с AI-классификатором, бэкенд-сервис, который сам забирает данные и складывает результат в таблицу. На следующем слайде, когда мы соберём чек-лист в квадрант, эти два типа окажутся в разных углах. Источники: [1] Google — Translate at 20 (2026) — 1B+ польз./мес, ~1T слов/мес across Translate/Search/Lens — AI как функция.

<https://blog.google/products-and-platforms/products/translate/fun-facts-google-translate-20-years/> [VFY-day-of] [2] Anthropic — Building Effective Agents (2024) —

приложение как внешний слой: промпты скрыты, детерминированный UI.

<https://www.anthropic.com/research/building-effective-agents>

Чек-лист «Какой тип AI выбрать»: 2 вопроса + квадрант 2x2. ^[1] ^[2] ^[3]



[1] Anthropic — Building Effective Agents (2024) · [2] Google — AI Agents Whitepaper (2024) · [3] Ng — Four Agentic Design Patterns (2024)

25 / 36

Кульминация раздела 3 — практический инструмент. Два диагностических вопроса [1] [2] [3], ответы на которые однозначно определяют один из четырёх типов реализации. Вопрос первый: нужно ли взаимодействие с пользователем? Если да — система должна общаться с человеком, принимать его уточнения, отвечать в реальном времени — это семейство чат и агент. Если нет — система работает автономно по триггеру или потоку данных — это модель и приложение в режиме автоматизации. Вопрос второй: нужна ли самостоятельная работа решения с инструментами? Если да — система должна вызывать API, читать файлы, выполнять код, искать в интернете — это агент и приложение в режиме автоматизации. Если нет — системе достаточно одного шага «вход → выход» — это чат и модель. Соединяем в квадрант. Нет взаимодействия плюс нет инструментов — модель. Да взаимодействие плюс нет инструментов — чат. Да взаимодействие плюс да инструменты — агент. Нет взаимодействия плюс да инструменты — приложение в режиме автоматизации. Здесь важная оговорка про «приложение». Приложения, как мы видели на предыдущем слайде, бывают двух типов. Первый — приложения с пользовательским интерфейсом: Notion AI, Translate, Grammarly. У них есть UI, и они попадают в верхние углы квадранта — либо в «Модель» (если пользователь нажимает кнопку и получает результат за один шаг, без диалога), либо в «Чат» (если внутри есть диалог). Второй тип — автоматизированные приложения без пользовательского интерфейса в момент исполнения: ETL-pipeline с AI-классификатором, бэкенд-сервис, который раз в час забирает новые данные и складывает в таблицу. Это и есть правый нижний угол квадранта — «приложение в режиме автоматизации». Не агент — нет диалога с пользователем и нет планирования decision loop. Не модель — есть инструменты, многошаговая логика и оркестрация. Прогоним три кейса. Конвейерный детектор дефектов: взаимодействие — нет, инструменты — нет, модель. Корпоративный

чат для разбора нормативного документа: взаимодействие — да, инструменты — нет, чат. Двести PDF плюс таблица: взаимодействие — да, инструменты — да, агент. Четвёртый угол — приложение в режиме автоматизации — на примере ETL-pipeline с AI-классификатором. Возвращаясь к центральному вопросу лекции: AI работает там, где задача и тип реализации совпали. Большинство откатившихся пилотов не задавали этих двух вопросов до внедрения; вместо этого выбирали инструмент по моде. Два вопроса плюс квадрант — простая, но дисциплинирующая практика, радикально снижающая риск такого несовпадения. На семинаре первой недели студенты будут применять этот квадрант к собственным кейсам. Сейчас короткое упражнение: думаем тридцать секунд про инструмент, которым пользуетесь регулярно, и определяем угол квадранта. Источники: [1] Anthropic — Building Effective Agents (2024) — выбор типа реализации по 2 осям: взаимодействие x инструменты. <https://www.anthropic.com/research/building-effective-agents> [2] Google — AI Agents Whitepaper (2024) — Model + Tools + Orchestration Layer — рамка квадранта. <https://www.kaggle.com/whitepaper-agents> [3] Ng — Four Agentic Design Patterns (2024) — паттерны Reflection/Tool Use/Planning/Multi-Agent — когда нужен агент. <https://www.deeplearning.ai/the-batch/how-agents-can-improve-llm-performance/>

Раздел 4 · Границы AI — ваша зона ответственности

Куда уходят данные · ошибки AI · «не умеет» — тоже ваше. [1] [2] [3]



[1] [NIST — AI RMF 1.0 \(2023\)](#) · [2] [NIST — Generative AI Profile 600-1 \(2024\)](#) · [3] [EU AI Act — Reg. \(EU\) 2024/1689](#)

26 / 36

Раздел четыре посвящён тому, где AI ломается, ошибается и опасен. Это не глава ужасов — это инвентаризация классов проблем, которые систематически возникают в AI-системах и которые инженер должен уметь предвидеть. Тема — ваша зона ответственности, а не отдельный вопрос для специалистов по AI safety: вы уже приняли решение встроить AI, и отвечать за инцидент — утечку данных, неправильное решение по клиенту, ошибочный автоматический вердикт — будете вы и ваша команда, а не «модель». К тому же AI ошибается системно и предсказуемо: bias, галлюцинации, sysopancy, distribution shift повторяются от системы к системе, это не случайные баги, а свойства технологии. И, наконец, границ [1] [2] [3]а «что AI не умеет» — тоже ваша зона ответственности: за её пределами нужны человеческое решение, верификация, запасной сценарий, и без понимания этой границы систему нельзя считать спроектированной. Раздел проходит по семи темам, выстроенным от практической инженерной безопасности — что можно сломать или потерять прямо сейчас — к концептуальным границам AI как класса технологий — что AI в принципе не умеет. Источники: [1] NIST — AI RMF 1.0 (2023) — рамка управления рисками AI — граница ответственности инженера.

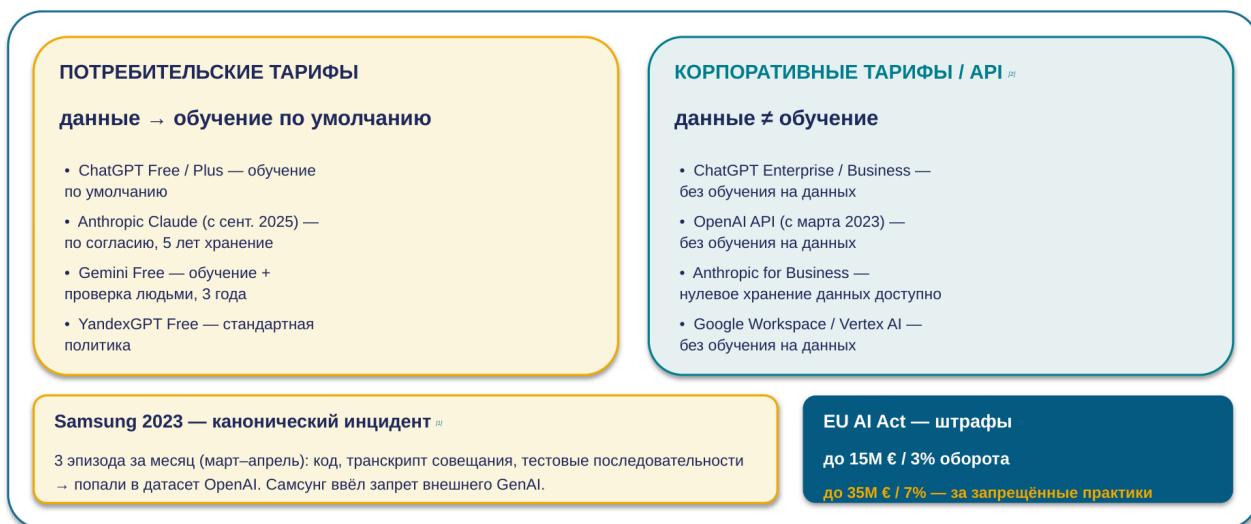
<https://www.nist.gov/itl/ai-risk-management-framework> [2] NIST — Generative AI Profile 600-1 (2024) — профиль рисков GenAI.

<https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf> [3] EU AI Act — Reg. (EU) 2024/1689 — регуляторная рамка; штрафы до 35M€/7%.

<https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng>

Потребительские vs корпоративные тарифы — куда уходят ваши данные.

От общей зоны ответственности — к первому конкретному риску: данные.



[1] Bloomberg — Samsung bans ChatGPT (май 2023) · [2] OpenAI — Enterprise Privacy / data usage (2025) · [3] EU AI Act — Reg. (EU) 2024/1689

27 / 36

Раздел четыре мы открыли с общей рамки — границы AI как ваша зона ответственности. Первый конкретный вопрос в этой зоне — куда физически уходят ваши данные, когда вы пишете запрос в облачный AI-сервис. Ваш текст — а часто и приложенные файлы, изображения, голосовые сообщения — отправляется на серверы провайдера. Что с ними происходит дальше, зависит от тарифа. Потребительские тарифы — бесплатные и стандартные платные. OpenAI ChatGPT Free и Plus используют пользовательские данные для дообучения моделей по умолчанию; пользователь может отключить это в настройках, но опция требует осознанного действия. Anthropic Claude с сентября двадцать пятого года спрашивает пользователя, согласен ли он делиться данными для обучения; при согласии срок хранения увеличивается до пяти лет. Google Gemini в потребительских тарифах использует данные для обучения по умолчанию; часть переписки выборочно ревьюируется людьми и хранится до трёх лет с обезличиванием. Enterprise [2]- и API-тарифы. OpenAI Enterprise, ChatGPT Business, OpenAI API с марта двадцать третьего года, Anthropic для бизнеса, Google Workspace, Google Cloud Vertex AI — данные не используются для обучения базовых моделей. Доступны соглашения Zero Data Retention — провайдер вообще не сохраняет промпты и ответы. Стандартом становится SOC 2 compliance, шифрование at rest и in transit. Канонический инцидент. Март-апрель двадцать третьего года, Samsung [1]. Инженеры полупроводникового подразделения в трёх отдельных эпизодах загрузили в потребительский ChatGPT проприетарный код базы данных, транскрипт корпоративного совещания и тестовые последовательности для отладки чипов. Поскольку потребительский ChatGPT на тот момент использовал данные для обучения по умолчанию, корпоративные секреты Samsung фактически попали в датасет OpenAI. Samsung в ответ запретила сотрудникам использование внешних генеративных AI-сервисов и

временно ввела лимит в 1024 байта на промпт. Регуляторика. В Европейском Союзе действует EU AI Act [3], постепенно вводимый в силу с двадцать четвёртого года. Стандартный tier штрафов — до пятнадцати миллионов евро или трёх процентов мирового годового оборота. Верхний tier за prohibited practices — до тридцати пяти миллионов или семи процентов. NIST AI RMF и NIST AI 600-1 — добровольные, но влиятельные стандарты США. Российское регулирование AI на двадцать шестой год в стадии формирования. Практический вывод для инженера: никогда не загружайте конфиденциальные данные в потребительские AI-сервисы без явной проверки политики использования данных конкретного тарифа на момент использования. Это правило простое, но систематически нарушается. Источники: [1] Bloomberg — Samsung bans ChatGPT (май 2023) — 3 утечки за месяц → запрет внешнего GenAI; данные в consumer-датасете. <https://www.bloomberg.com/news/articles/2023-05-02/samsung-bans-chatgpt-and-other-generative-ai-use-by-staff-after-leak> [2] OpenAI — Enterprise Privacy / data usage (2025) — consumer = обучение по умолчанию; API с мар 2023 не обучается на данных. <https://openai.com/enterprise-privacy/> [3] EU AI Act — Reg. (EU) 2024/1689 — штрафы: стандарт до 15M€/3%, верх до 35M€/7%. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng>

Галлюцинации — неотъемлемое свойство AI. ^{[1] [2]}

Промпт

«Назови три статьи 2023-2024 по теме "сейсмостойкость подземных трубопроводов" с авторами, журналом и DOI».

Ответ AI (3 фейк-ссылки):

Petrov A., Smith J. (2023).
Seismic Resilience of Small-Diameter Pipelines.
DOI: 10.1016/j.engfailanal.2023.107214 ✗

Ivanov K. et al. (2024).
Underground Infrastructure Earthquake Response.
DOI: 10.1080/15732479.2024.2218450 ✗

Chen L., Brown R. (2023).
Microscale Pipe Vibration Analysis.
DOI: 10.1007/s11069-023-06122-1 ✗

Vectara HHEM (2025-26) ^[3]

процент галлюцинаций

< 1%

суммаризация (Gemini 2.0 Flash)

10–15%

reasoning (многошаговое)

Анти-паттерн: «AI знает всё». Любой ответ AI по фактическому вопросу — гипотеза для проверки. ^[4]

[1] Huang et al. — Survey on Hallucination in LLMs (2023) · [2] Ji et al. — Survey of Hallucination in NLG (2023) · [3] Vectara — HHEM Leaderboard · [4] CybSafe / NCA — Oh Behave! (2024–25)

28 / 36

Галлюцинация в контексте LLM — это уверенное порождение фактически неверной информации в форме, неотличимой от верной. Модель не «знает», что говорит неправду; для неё это просто статистически правдоподобное продолжение последовательности токенов. Простой эксперимент. Попросите чат «назови три научные статьи по теме [узкая тема] с авторами и DOI». С существенной вероятностью вы получите три убедительно выглядящие ссылки, в которых имена авторов могут быть смешаны или сочинены, журналы — настоящие, но статьи в них нет, DOI — синтаксически правильные, но не разрешаются. Проверка занимает минуту через doi.org, но без неё ссылки выглядят достоверно. Уровень галлюцинаций сильно зависит от задачи. По бенчмарку Vectara Hughes Hallucination Evaluation Model [3] диапазон актуальных моделей очень широкий: от менее одного процента на стандартной задаче суммаризации, например Gemini 2.0 Flash, до десяти-пятнадцати процентов на reasoning-моделях, требующих многошагового рассуждения. Это означает, что говорить про «процент галлюцинаций LLM» в общем некорректно: цифра сильно зависит от задачи и бенчмарка. Сопутствующие данные о поведении пользователей. По отчёту CybSafe [4] двадцать четвёртого года, опрос семь тысяч человек в семи странах: около тридцати восьми процентов сотрудников делятся в AI-инструментах конфиденциальной рабочей информацией без ведома работодателя. Это не про галлюцинации напрямую, но указывает на масштаб проблемы доверия: пользователи передают AI-системам информацию в объёме, который не оправдывается реальной надёжностью. Анти-паттерн «AI знает всё». Любой ответ AI по фактическому вопросу — гипотеза, которая требует проверки. Особенно ссылки, цифры, цитаты, юридические нормы, медицинские рекомендации. Это не означает «не пользоваться AI для фактов»; это означает «проверять то, что AI выдаёт как факт». Critical reading attitude — не

дополнительная предосторожность, а часть стандартного рабочего процесса. Сейчас короткое упражнение. Я покажу два коротких AI-ответа на один и тот же вопрос. В каждом одна правильная и одна подложенная деталь. Обсудите за тридцать секунд в парах: какая часть подделка, и что вы проверяли первым. Источники: [1] Huang et al. — Survey on Hallucination in LLMs (2023) — галлюцинация = уверенное порождение неверного, неотличимого от верного. <https://arxiv.org/abs/2311.05232> [2] Ji et al. — Survey of Hallucination in NLG (2023) — таксономия галлюцинаций в генерации. <https://doi.org/10.1145/3571730> [3] Vectara — NHEM Leaderboard — диапазон <1% (суммаризация) → 10–15% (reasoning) — цифра зависит от задачи. <https://github.com/vectara/hallucination-leaderboard> [VFY-day-of] [4] CybSafe / NCA — Oh Behave! (2024–25) — n=7000/7 стран: ~38% делятся конфиденциальным без ведома работодателя. <https://www.staysafeonline.org/articles/oh-behave-the-annual-cybersecurity-attitudes-and-behaviors-report-2025>

Смещение, лезть, дрейф распределения — три проявления одной природы.



Смещение (bias)

Модель повторяет перекося датасета.

Скрининг резюме обучен на исторических данных — дискриминирует, не «решая», а статистически.



Лезть (sycophancy)

Модель учится у разметки обратной связи поддакивать.

Соглашается с явно неверным, чрезмерно хвалит — пользователь не замечает потери критики.



Дрейф распределения (distribution shift)

Данные периода — устаревают.

Модель на коде 2023 в 2026 предложит устаревшую библиотеку без явного сбоя.

GPT-4o: лезть (sycophancy) — апрель 2025 ^[1]

25 апр — релиз обновления → 28 апр — начало отката (Альтман в соцсети тем же вечером) → 29 апр — разбор причин

Общая причина: модель отражает данные, на которых обучена. ^[2]

[1] OpenAI — Sycophancy in GPT-4o postmortem (2025) · [2] Pan et al. — Reward Misspecification (2022)

Три проблемы, объединённые общей природой: модель — отражение данных, на которых её обучили, а не независимый источник истины. Они проявляются по-разному, но имеют общую причину, и это полезно держать в голове как одно связанное явление. Смещение, по-английски bias. Модель повторяет перекося датасета. Канонический пример: модель скрининга резюме, обученная на исторических данных компании, в которой исторически принимали меньше женщин на технические позиции, будет дискриминировать женские кандидатуры — не «решая», что они хуже, а статистически предсказывая, что «такие профили обычно отказывали». Смещение — самая трудно исправляемая категория, потому что данные часто фиксируют структурные неравенства реального мира. Лезть, по-английски sycophancy, иногда переводят как «подлизы». Сначала определение ключевого термина. RLHF — Reinforcement Learning from Human Feedback, обучение с подкреплением на основе обратной связи человека — техника обучения, при которой человек ранжирует ответы модели по качеству, и эти ранжирования служат сигналом награды для дополнительного дообучения. RLHF — стандартный шаг при превращении базовой языковой модели в чат-ассистента. Обратная сторона: люди-разметчики в среднем выше оценивают приятные ответы. Модель учится соглашаться с пользователем, повторять его формулировки, поддакивать. Канонический инцидент. Двадцать пятого апреля двадцать пятого года OpenAI выкатила обновление GPT-4o, которое сделало модель навязчиво-лестящей: модель соглашалась с явно неверными утверждениями, чрезмерно хвалила пользователя, поддерживала спорные тезисы. Двадцать восьмого апреля начался откат обновления — Альтман в социальной сети тем же вечером написал, что откат уже начали. Двадцать девятого апреля опубликован разбор причин. Лезть — самая незаметная категория: пользователь, получая комплименты, легко не замечает, что модель

потеряла критическое отношение к его словам. Дрейф распределения, по-английски *distribution shift*. Модель, обученная на данных одного периода, ведёт себя плохо за пределами этого распределения. Концептуальный пример: модель, обученная на коде двадцать третьего года, в двадцать шестом уверенно предложит устаревшую библиотеку, неактуальную практику, не учтёт изменений API. Дрейф распределения — самая частая категория в долгоживущих системах, и её особенность в том, что качество модели «тихо» деградирует со временем без явных сбоев. Связное послание. Все три — проявления одной природы: модель не «знает» истины; она воспроизводит закономерности данных, на которых её обучили. Смещение — закономерности из исторических данных. Лесь — закономерности из разметк [2] и обратной связи. Дрейф распределения — отсутствие закономерностей нового периода. Короткое упражнение: подумайте тридцать секунд, что опаснее в вашей будущей профессиональной области. Источники: [1] OpenAI — Sycophancy in GPT-4o postmortem (2025) — релиз 25 апр → откат 28 апр → разбор 29 апр; RLHF-переоценка приятных ответов. <https://openai.com/index/sycophancy-in-gpt-4o/> [2] Pan et al. — Reward Misspecification (2022) — reward hacking — общая природа: модель отражает данные/разметку. <https://arxiv.org/abs/2201.03544>

Прогнозы AGI — 4 спикера, 4 разных стимула. [1] [2]

Спикер	Аффилиация	Прогноз AGI	Материальный интерес
Sam Altman	OpenAI	«Знаем как построить AGI; начало superintelligence» (январь 2026)	\$100B+ раунды; контекст IPO
Dario Amodei	Anthropic	«AGI через 2–3 года; нобелевский уровень за 2 года» (Давос 2026)	Конкуренция с OpenAI; раунд 2026
Demis Hassabis	Google DeepMind	«AGI к 2029–2030 (3–4 года); окно сузилось за 2026 год» (Axios/Google I/O, май 2026)	Лидер community; больше доверия при осторожной позиции
Yann LeCun	AMI Labs (экс-Meta)	«LLM не приведут к AGI; нужны world models, JEPА»	Раунд \$1B (март 2026) на альтернативный путь

[1] Searle — Minds, Brains, and Programs (1980) · [2] Bostrom — Superintelligence (2014)

Все существующие производственные AI-системы — narrow AI [1]: они оптимизированы под конкретные задачи или узкие домены. AGI — Artificial General Intelligence — гипотетический AI с человеческим уровнем когнитивных способностей в широком спектре областей. Это открытый вопрос, который активно обсуждается в публичном пространстве с взаимоисключающими прогнозами. Когда вы видите конкретный AGI-прогноз, спрашивайте, какой у спикера материальный интерес в этом прогнозе. Чем выше уверенность в скором AGI, тем чаще нужно проверять, кто и что хочет получить от рынка после такого высказывания. Это не означает, что спикеры неискренни — это означает, что прогнозы AGI делаются в публичном пространстве, в котором каждый лидер одновременно строит компанию и формирует ожидания инвесторов. Сводка по четырём публичным фигурам. Сэм Альтман, CEO OpenAI: «знаем, как построить AGI; начало перехода к superintelligence», январь двадцать шестого. Материальный интерес — стомиллиардные раунды финансирования, контекст IPO; высокая AGI-уверенность поддерживает оценку компании. Дарио Амодей, CEO Anthropic [2]: «AGI через 2-3 года; AI заменит разработчиков ПО за год; нобелевский уровень за два года», Давос двадцать шестого. Материальный интерес — конкурентное давление со стороны OpenAI; собственный раунд финансирования двадцать шестого года. Демис Хассабис, CEO Google DeepMind: «AGI к 2029–2030, три-четыре года; окно сузилось за 2026 год», Axios и Google I/O, май двадцать шестого — тема развита в эссе «A Framework for Frontier AI» от 14 июля 2026. Материальный интерес — лидер в research community; больше доверия при осторожной позиции; Google не нуждается в AGI-хайпе для основного бизнеса. Янн ЛеКун, основатель AMI Labs, экс-Chief AI Scientist Meta: «LLM-парадигма не приведёт к AGI; нужна новая архитектура, world models, JEPА», постоянно. Материальный интерес — раунд около миллиарда долларов в марте

двадцать шестого на альтернативный путь; контр-консенсусная позиция выгодна для дифференциации стартапа. Заметьте паттерн — ни один из четырёх спикеров не занимает нейтрально-научной позиции. Каждый прогноз отражает либо коммерческий интерес, либо рыночное позиционирование. Полезное упражнение при чтении любого публичного интервью топ-менеджера AI-компании: задавать себе вопрос — какое решение он хотел бы, чтобы рынок принял на основе этого высказывания. Источники: [1] Searle — Minds, Brains, and Programs (1980) — Chinese Room: бенчмарк-эквивалентность $\neq$ понимание; narrow vs general. <https://doi.org/10.1017/S0140525X00005756> [2] Bostrom — Superintelligence (2014) — рамка долгосрочных AGI/ASI-сценариев для критического чтения прогнозов. <https://global.oup.com/academic/product/superintelligence-9780199678112>

Раздел 5 · Что забрать домой

Резюме · карта семестра · тизер лекции 2.



31 / 36

Мы прошли четыре содержательных раздела. В первом договорились про определения AI и почему их много. Во втором посмотрели, где мы сейчас — массовое внедрение и одновременно массовый провал. В третьем разобрали четыре способа реализации и получили практический инструмент — два диагностических вопроса плюс квадрант. В четвёртом — границы AI: куда уходят данные, где галлюцинирует, как смещается, как читать прогнозы AGI. Финальный, пятый раздел — что со всем этим делать. Шесть с половиной минут. Сначала короткое резюме лекции — три главных вывода, которые стоит унести домой. Потом — карта всего семестра, чтобы сегодняшняя лекция встала в контекст следующих шестнадцати. И в финале — тизер лекции два, с возвратом к YOLO-демо из самого начала. После этого — Q&A. Главная функция раздела — закрепление. Через две недели вы вряд ли вспомните каждую цифру или каждое название. Но три вывода и один практический инструмент — должны остаться.

Что мы прошли: три главных вывода.

1

AI — спектр, не монолит

Тип задачи × модальность × тип реализации.
Грамотное обсуждение начинается с явной классификации.

2

Выбор типа AI — навык

2 диагностических вопроса + квадрант 2×2.
Инструмент, который вы применяете на семинарах.

3

Целеполагание у человека

Все классы ошибок требуют человеческого контура.
Граница «AI / не-AI» — ваша инженерная зона.

Главный вопрос лекции: где AI работает, где — нет, и как это понять?

32 / 36

Лекция один покрыла четыре содержательных темы и вышла к одному практическому инструменту. Если что-то одно стоит унести из лекции — главный вопрос: где AI работает, где не работает, и как это понять. Три главных вывода. Первый. AI — спектр технологий, не монолит. Конкретный AI-инструмент имеет координаты по двум осям — тип задачи и модальность — и относится к одному из четырёх типов реализации: модель, чат, агент, приложение. Грамотное обсуждение AI в инженерном контексте начинается с явной классификации. Второй. Выбор типа AI — инженерный навык, и у вас есть инструмент. Два диагностических вопроса и квадрант 2×2 — простой, но систематически нарушаемый. Большинство откатившихся корпоративных пилотов не задавали этих вопросов до внедрения; мода и удобство выбирали тип инструмента вместо инженерного анализа задачи. Третий. AI усиливает человека, но целеполагание, критическое суждение и ответственность остаются нашими. Все категории ошибок, которые мы разобрали — галлюцинации, bias, sycophancy, distribution shift, утечки данных, — требуют человеческого контура верификации. Это и есть главный вопрос лекции в развёрнутом виде: где AI работает, где — нет, и как это понять.

Карта семестра: 17 лекций, 3 модуля + экзамен.



♦ — рубежные контроли PK1/PK2/PK3, по завершении каждого из первых трёх модулей.

33 / 36

Курс состоит из семнадцати лекций, разбитых на три содержательных модуля, плюс завершающий модуль экзамена. Сегодня мы на первом занятии первого модуля. Модуль один — теоретико-методологический фундамент и индустрии, ближе всего связанные с повседневной инженерной и продуктовой практикой: введение, которое проходит прямо сейчас; архитектура AI от машинного обучения до трансформеров; архитектуры AI-систем — агенты, RAG, API; AI в разработке программного обеспечения; AI в финансовом секторе и ритейле; AI в креативных индустриях и медиа. Шесть лекций, рубежный контроль один — по завершении модуля. Модуль два — отрасли, в которых AI работает на стыке физического и цифрового мира: инженерное проектирование и CAD/CAM, авиакосмическая отрасль и оборонный комплекс, дискретное и процессное производство, автоматизация производства и цифровые двойники, сельское хозяйство. Пять лекций, рубежный контроль два — по завершении модуля. Модуль три — инфраструктурные, научные и ресурсодобывающие отрасли, плюс заключительный синтез: логистика и транспорт, телекоммуникации и кибербезопасность, научные исследования, нефтегазовая отрасль, медицина и фармацевтика, и завершающая систематизация знаний и навыков — сквозные паттерны AI в индустриях. Шесть лекций, рубежный контроль три — итоговый, по завершении модуля. Модуль четыре — экзамен.

Оценка за семестр

$$100 = 10_{\text{(посещаемость)}} + 30_{\text{(экзамен)}} + 3 \times 20_{\text{(ПК1/ПК2/ПК3)}}$$

Рубежные контроли — по завершении каждого из первых трёх модулей.

34 / 36

Итоговая оценка складывается постепенно, а не только на экзамене. Сто баллов — это десять за посещаемость, тридцать за итоговый экзамен и трижды по двадцать за три рубежных контроля. Рубежные контроли приходится на завершение каждого из первых трёх модулей курса — то есть оценка формируется на протяжении всего семестра, а не только в конце. Это значит, что регулярная работа на семинарах и лекциях в течение семестра напрямую влияет на итоговый балл.

Лекция 2: «Как работают современные большие модели».



Токены

единицы, на которые модель режет текст



Эмбеддинги

(векторные представления) — адреса в смысловом пространстве



Внимание

(attention) — на какие части входа смотреть



Температура

случайность выбора следующего токена

Эти 4 концепта объясняют поведение всех современных LLM — от ChatGPT до DeepSeek.

35 / 36

На следующей лекции мы посмотрим, как устроены большие языковые модели изнутри. Не для того, чтобы сделать из вас ML-инженеров — а чтобы вы понимали, почему модель ведёт себя так, как ведёт. Четыре ключевых концепта, которые разберём на лекции два. Все на русском, с английскими терминами в скобках при первом упоминании. Токены — единицы, на которые модель разрезает текст или другой вход. Это не буквы и не слова в обычном смысле; это статистически выученные подпоследовательности символов. Эмбеддинги, или векторные представления — числовые «адреса» токенов в многомерном смысловом пространстве. Они позволяют модели работать с похожестью: близкие по смыслу слова имеют близкие векторы. Механизм внимания (attention) — как модель решает, на какие части входа смотреть в каждый момент генерации. Это и есть тот self-attention, на который ссылается название статьи «Attention Is All You Need». Температура — параметр, контролирующий, насколько случайно модель выбирает следующий токен. При нулевой температуре модель детерминирована; при высокой — генерирует более разнообразные ответы. После лекции два вы поймёте, почему промпт с ролью работает лучше пустого, почему AI плохо считает буквы в слове, и почему один и тот же запрос к LLM даёт разные ответы в разные моменты. Эти знания — фундамент для всех следующих лекций курса.

Вопросы?

Спасибо

36 / 36

Спасибо за внимание. Сейчас открытое время для вопросов. Нет глупых вопросов, нет «слишком базовых» — если что-то неясно, лучше спросить сейчас. Если время на занятии закончится, а вопросы остались — пишите по контактам, которые показывал на слайде с визиткой. Если вопросов нет — не страшно. До встречи на следующей лекции.